# Micro-Harmony

Cormac Flanagan

August 19, 2008

**Abstract**

This paper formalizes some ideas underlying ES-Harmony, in particular the unified language for both types and terms, and the idea of type annotations as dynamic checks.

## 1 Language Overview

We consider the implementation of a *gradual typed* language that supports both typed and untyped terms, which interoperate in a flexible manner. We begin by defining the syntax of terms and types in the language: see Figure 1. The syntax unifies terms and types. The language extends the lambda calculus (variables, abstractions, and application) with object allocation, dereference, and update. It also includes `as` expressions, which check that a value has a particular type. Object addresses $a_T$ should not occur in source programs.

We use $e$ and $E$ as meta-variables ranging over expressions (aka terms), where $E$ indicates that the expression is intended to evaluate to a type. The meta-variables $S$ and $T$ indicate *type values*, or simply, *types*.

The type language includes integers, function types, object types, and the type $*$, which indicates that no type information is known.

**Figure 1: Syntax**

| | | |
|---|---|---|
| $e, E ::=$ | | *Terms and types:* |
| | $n$ | integer constants |
| | $b$ | boolean constants |
| | $x^c$ | variable |
| | $\lambda x\!:\!E.\ e : E'$ | abstraction |
| | $(e\ e)$ | application |
| | $\{\bar{l} = \bar{e}\} : E$ | object allocation |
| | $e.l$ | member selection |
| | $e.l := e$ | member update |
| | $e$ `as` $E$ | runtime type check |
| | $a_T$ | object address |
| | `Int` | integer type |
| | `Bool` | boolean type |
| | $E \to E$ | function type |
| | $\{\bar{l} : \bar{E}\}$ | object types |
| | $*$ | dynamic type |
| $S, T ::=$ | | *Type Values:* |
| | `Int` | integer type |
| | `Bool` | boolean type |
| | $S \to T$ | function type |
| | $\{\bar{l} : \bar{T}\}$ | object types |
| | $*$ | dynamic type |

## 2 Type Relations

**Figure 2: Subtyping and Compatibility**

Subtyping $\boxed{S < T}$

$$\frac{}{T < T} \quad \text{[Sub-Refl]}$$

$$\frac{S_1 = T_1 \qquad S_2 < T_2}{(S_1 \to S_2) < (T_1 \to T_2)} \quad \text{[Sub-Arrow]}$$

$$\frac{S_i = T_i \qquad \text{for } i \in 1..n}{\{l_i : S_i^{i \in 1..n+m}\} < \{l_i : T_i^{i \in 1..n}\}} \quad \text{[Sub-Obj]}$$

Compatibility $\boxed{S \twoheadrightarrow T}$

$$\frac{}{T \twoheadrightarrow *} \quad \text{[Com-Dyn]}$$

$$\frac{}{T \twoheadrightarrow T} \quad \text{[Com-Refl]}$$

$$\frac{S_1 \twoheadrightarrow T_1 \qquad S_2 \twoheadrightarrow T_2}{(S_1 \to S_2) \twoheadrightarrow (T_1 \to T_2)} \quad \text{[Com-Arrow]}$$

$$\frac{S_i \twoheadrightarrow T_i \qquad \text{for } i \in 1..n}{\{l_i : S_i^{i \in 1..n}\} \twoheadrightarrow \{l_i : T_i^{i \in 1..n}\}} \quad \text{[Com-Obj]}$$

The type system is based in part on the usual subtype relation $S < T$. We write $S = T$ to mean that $S < T$ and $T < S$. For technical reasons, function subtyping is *invariant* in the argument position.

In addition, to support a notion of gradual typing, we also have a compatibility relation $S \twoheadrightarrow T$, which holds provided that $S$ and $T$ are identical except that $T$ may include $*$ in places where $S$ does not. The compatibility relation is asymmetric since $\texttt{Int} \twoheadrightarrow *$ but $* \not\twoheadrightarrow \texttt{Int}$, but is transitive.

The *compatible-subtyping* relation $S \lessdot T$ is then the composition of these two relations: $S \lessdot T$ holds if there exists $U$ such that $S < U$ and $U \twoheadrightarrow T$.

**Lemma 1** *Subtyping is transitive.*

**Lemma 2** *Compatibility is transitive.*

**Lemma 3** *Subtyping and compatibility commute, that is,* $(< \circ \twoheadrightarrow) = (\twoheadrightarrow \circ <)$.

PROVEME.

**Lemma 4** *Compatible-subtyping is transitive.*

We have:

$$\begin{array}{rcl} (S \to T) & \lessdot & (* \to *) \\ \{l : T, \ldots\} & \lessdot & \{l : *\} \\ \{\ldots\} & \lessdot & \{\} \end{array}$$

# 3 Evaluation

We next describe the evaluation semantics of the language. The set of values in the language is given by:

| $v ::=$ | | *Values:* |
|---|---|---|
| | $n$ | integer constant |
| | $b$ | boolean constant |
| | $\lambda x{:}S.\ e : T$ | abstraction |
| | $a_T$ | object address with type $T$ |
| | $T$ | type value |

Each object address $a$ is annotated with the type $T$ of the object it points to. A *object store* $\sigma$ maps object addresses $a_T$ to object values of the form $\{\bar{l} = \bar{v}\}$. Values and object values in the store are *closed* in that they do not contain free program variables $x$; though they may contain object addresses.

Every value has an *allocated type* according to the function $ty(v)$:

$$\begin{array}{rcl} ty(n) & = & \texttt{Int} \\ ty(b) & = & \texttt{Bool} \\ ty(\lambda x{:}S.\ e : T) & = & (S \to T) \\ ty(a_T) & = & T \\ ty(T) & = & * \end{array}$$

The allocated type of an object is invariant and independent of the store $\sigma$. A *type tag* is a type that can be returned by $ty(v)$; it includes `Int`, function, and object types, but excludes $*$. We use the notation $T.l$ to denote $S$ if $T = \{l : S, \ldots\}$, and to denote $*$ if $T$ is a different object type; in all other cases $T.l$ is undefined.

An evaluation context is:

$$\begin{array}{rcl} C & ::= & \bullet \mid (\ t) \mid (v\ ) \mid \ \texttt{as}\ T \mid \{\bar{l} = \bar{v}, l = C, \bar{l} = \bar{e}\} : T \mid .l \mid .l := e \mid v.l := \\ & & \mid \lambda x{:}.\ e : E \mid \lambda x{:}T.\ e : \ \mid \{\ldots\} : \ \mid \ \to E \mid T \to \ \mid \{l : S, \ldots, l' : C, \ldots, l'' : E, \ldots\} \mid e\ \texttt{as} \end{array}$$

A *state* is a pair of an object store and a current expression. The evaluation relation on states is defined by the rules in Figure 3. These rules perform dynamic type checks by calling the function $convert_\sigma(v, T)$, which checks if the value $v$ can be converted to the type $T$.

The notation $\sigma[a_T, l := v]$ denotes the store that is identical to $\sigma$, except that the $l$ field of the object at address $a_T$ is updated with the value $v$.

**Figure 3: Operational Semantics**

**Evaluation Rules**

| | | | | |
|---|---|---|---|---|
| $\sigma, C[((\lambda x{:}S.\ t : T)\ v)]$ | $\longrightarrow$ | $\sigma, C[t[x := v']\ \texttt{as}\ T]$ | if $v' = convert_\sigma(v, S)$ | [E-Beta] |
| $\sigma, C[v\ \texttt{as}\ T]$ | $\longrightarrow$ | $\sigma, C[v']$ | if $v' = convert_\sigma(v, T)$ | [E-As] |
| $\sigma, C[\{l_i = v_i^{i \in 1..n}\} : T]$ | $\longrightarrow$ | $\sigma[a_T := (\{l_i = v'_i\})], C[a_T]$ | if $v'_i = convert_\sigma(v_i, T.l_i)$, $a_T$ fresh, $T = \{\ldots\}$ | [E-Alloc] |
| $\sigma, C[a_T.l]$ | $\longrightarrow$ | $\sigma, C[v]$ | if $\sigma(a_T) = \{l = v, \ldots\}$ | [E-Get] |
| $\sigma, C[a_T.l := v]$ | $\longrightarrow$ | $\sigma[a_T, l := v'], C[v]$ | if $v' = convert_\sigma(v, T.l)$ | [E-Assign] |

**Dynamic Type Checks**

$$\begin{array}{rcll} convert_\sigma(v, T) & = & v & \text{iff } ty(v) <\!\!* T \\ convert_\sigma(0, \texttt{Bool}) & = & \texttt{false} & \\ convert_\sigma(n, \texttt{Bool}) & = & \texttt{true} & n \neq 0 \end{array}$$